Building an Alpenglow-Era System for MEV Identification in Solana Blocks

A Heuristic Informed Isolation Forest Approach to Detecting Suspicious Transaction Activity in Solana Blocks

Milan Suri

November 2026

1 Executive Summary

This paper introduces the idea of using a heuristic to guide tree splitting for an isolation forest. An isolation forest is an unsupervised machine learning algorithm that is sometimes used for anomaly detection. Instead of using the basic implementation of an isolation forest, we utilize a criterion scoring method to prioritize specific features to guide tree splitting. This allows the algorithm to both follow a rule based patterns and revert to random-splitting to find non-covered trends in normal order flow. As python is too slow to run on intra-block data for Solana (has to be performed in a 150ms time span before block finality due to Alpenglow), this model will be exported as an ONNX and then integrated into a C++ stack to improve processing speed.

2 Background

2.1 Isolation Forests

Isolation forests are an unsupervised machine learning algorithm that adopts a tree-based structure for anomaly detection. Traditional isolation forests work by random partitioning, where the algorithm will choose a random feature and then splits the feature at a random value dividing it into two parts and recursively does this, creating binary trees representing a split in the data. The number of splits for a data point is called an isolation path, and an anomaly is determined based on the length of the isolation path; as to isolate an anomaly there will be a shorter path as it differs from the data. This splitting is conducted over multiple trees to ensure reliability. Then averages the isolation path across the ensemble to produce an anomaly score, then a threshold will determine whether to classify that item as an anomaly or normal behavior.

2.2 Solana

Solana is an L1 blockchain built for speedy transactions and a low cost. Solana is divided into 400ms units called slots. These slots are then verified and processed through a lead validator chosen ahead of time to validate the transactions in that slot and publish them. Unlike Ethereum, Solana does not employ a global memory pool for the transaction waiting, instead it utilizes a program called Gulf Stream. Gulf Stream utilizes something called Remote Procedure Call Nodes (RPC nodes).

Solana is built using a Proof of Stake architecture, where each validator has a reserve of tokens that acts as its bid to be the lead validator. This then increases the likelihood that it'll be scheduled. Unlike validator nodes, RPCs have no stake. They will receive a transaction message and then relay it to the slot leader to be processed. This is called a gossip protocol and changes how transaction ordering on Solana works by using the gossip protocols to randomly propagate the transactions to different leader blocks across the network.

Then each validator once selected as a leader will process these transactions and broadcast blocks to the rest of the network to be voted on.

This voting process and block finality has recently changed with the release of Alpenglow on Solana mainnet-beta. Alpenglow reduces the block finality of Solana from approximately 12.8 seconds to 150 ms. This change reduces the voting process for Solana block finality by moving votes from validators off-chain, replacing TowerBFT and Proof of History with Votor and Rotor for changing the vote bundling and data propagation on the network.

2.3 MEV

Maximal Extractable Value (MEV) is the concept of the maximum value that can be generated from block production on a blockchain like Solana by including, excluding, or reordering transactions within a block. This is usually carried out by validators or automated trading algorithms and common examples include sandwich attacks and front-running on exchanges.

Front-running works on Solana by seeing a pending purchase in the transaction queue and then using a priority fee to get ahead in the transaction processing order, moving the price, and then selling to the purchaser for a higher price. Another example of MEV is sandwich attacks is when a bot places an order right before another order for that asset to drive the price up and then follow that up to sell the asset at a profit.

As Alpenglow has reduced the block finality time of Solana, therefore shifting the amount of time an algorithm has to visibility within the block before publication, shrinking the window for priority or validator re-ordering. This has increased the need for the use of Jito bundles or faster transactions and co-location for being able to successfully conduct MEV on the new finality protocol.

Jito is an infrastructure protocol on Solana for optimizing performance and providing MEV opportunities. Two important Jito features for MEV are the Jito-Solana client is an open-source validator client to help validators receive maximize receive more rewards by running an auction for transaction ordering, and Jito bundles a feature allowing bots to submit an atomic order (all or nothing) for up to 5 transactions to be executed sequentially in the same slot.

Additionally, colocation is a difficult problem on Solana, as it is a decentralized platform, meaning that as the validator changes every 400ms and are located globally, there isn't a centralized platform to connect a chip to.

3 Proposal

3.1 Problem

MEV is an issue for Solana as it creates a hostile environment for less established participants to engage in.

3.2 Proposed Solution

To better prevent MEV on Solana, you need to begin with a way of detecting it. The premise of this paper and project are to establish a way of doing that through the use of ML and fast execution.

As the information of a block before it is published is limited, but each column provides valuable data, there needs to be an inherent weighing system to provide a heuristic to guide proper or rational splitting of trees, rather than a purely random architecture. This is because certain identifiers like transaction ordering, compute unit behavior, and account interactions all can be different from normal account activity on Solana when MEV is present. Therefore, by being able to define the parameters of what normal behavior should look like on Solana by using past data to establish weighing, the tree should be better used to identify suspicious activity.

This idea is based on the principles of game-playing reinforcement learning models, as they begin training by defining the constraints for the game. This essentially creates the rule-engine by dictating normal order flow. By making the model guided by what normal activity should look like, this will theoretically allow for the model to adapt to newer methods of MEV as they appear. Additionally, it should be able to weigh the relevant features to produce an anomaly score, so it follows the established behavior or history of the network. This also prevents the need for creating multiple different constraint sets for different types of MEV, by being able to capture overall suspicious behavior.

While isolation forests are fast, they won't be able to process the blocks incoming data in the 150 ms block finality time that Alpenglow establishes. This would be addressed through the porting of the model to ONNX, an open-source framework for representing ML models. Through ONNX runtime, we're able to optimize the model performance by applying better constant folding, operator fusion, and memory planning. In addition to exporting the isolation forest from python to ONNX, we will import the model into C++ to avoid Python's GIL.

Once imported into C++ we'd implement lock free queues for storing all non-vote transactions in a block and then using a pinned thread pool for iterating over each transaction and running it on the isolation forest. Additionally, in C++ we'd use a compile-time architecture utilizing template meta programming to create compile-time generalized structures for representing the data ingested.

To ingest intra-block data, we'd utilize originally a webhook from Helius LaserStream, pivoting to gRPC for faster throughput once proof of concept is established. Additionally, to bypass OS jitter, DPDK would be utilized to stream ingested data from the NIC directly to the program.

Once the transactions are assessed by the model, they will produce an anomaly score that will be interpreted into whether they are MEV or normal transaction behavior. This will originally be assessed on synthetic data in python to ensure that the model is functional before moving to C++ and integrating into a low-latency architecture. This will allow us to get statistics for accuracy since finalized block data will not tell us if a transaction was MEV.

4 Transparency

The main concern of this system is its interoperability and transparency. How do you make a black box model transparent, so one can audit the results? Off the hot path, SHAP values for each block will be generated and stored so a user can examine the biases of the model and how the transaction was evaluated resulting in a final score.

5 Current Results and Work

Got real data and generated synthetic data and then started using it to develop the heuristic weights and train an isolation forest.

Used Helius webstream to get recent block data and download to disk. Then used a CTGANSynthesizer to generate 10,000 synthetic transactions on the total dataset of 1223 real transactions. Did statistic summary of the dataset and then calculated the feature weights based on the real data. Then fed into the isolation forest using the weights. It's a tiny version would be great to get 1 million+ real transactions.

Table 1: Normalized Feature Weights for MVP

Feature	**Normalized Weight**
Compute Units	0.1360
Total Fee (Lamports)	0.2361
Account Count	0.1550
Instruction Count	0.1625
Inner Instruction Count	0.1472
Token Balance Change Count	0.1632

Table 2: Weighted Isolation Forest Performance Comparison

(a) Real Dataset

Metric	**Value**
Transactions	245
Anomalies	14
Anomaly Rate (%)	5.71
Mean Score	0.3974
Median Score	0.3644
95th Percentile	0.5550
Maximum Score	0.6970

(b) Synthetic Dataset

Metric	**Value**
Transactions	10,000
Anomalies	652
Anomaly Rate (%)	6.52
Mean Score	0.4393
Median Score	0.4263
95th Percentile	0.5530
Maximum Score	0.6559

Check out the github to see current code:
`https://github.com/MilanSuri/CSC561Final`

6 Citations

Maximal Extractable Value (MEV)

[1] Jito Labs. "Jito-Solana Features," Jito Labs Validator Documentation, Accessed: Nov. 8, 2026. [Online]. Available: https://jito-foundation.gitbook.io/mev/jito-solana/features

[2] Jito Labs. "What is Jito?" Jito Labs Documentation for Searchers, Accessed: Nov. 8, 2026. [Online]. Available: https://docs.jito.wtf/

[5] Lostin. "Solana MEV Report: Trends, Insights, and Challenges," Helius, Accessed: Nov. 8, 2026. [Online]. Available: https://www.helius.dev/blog/solana-mev-report

Solana

[4] Lostin. "Solana Executive Overview," Helius, Accessed: Nov. 8, 2026. [Online]. Available: https://www.helius.dev/blog/solana-executive-overview

[6] A. Yakovenko, "Solana: A New Architecture for a High Performance Blockchain v0.8.13," Solana, White Paper, Nov. 2017. [Online]. Available: https://solana.com/solana-whitepaper.pdf

Isolation Forests

[3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-Based Anomaly Detection," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 1, pp. 1–39, 2012. DOI: 10.1145/2133360.2133363 [Online]. Available: https://doi.org/10.1145/2133360.2133363